Data Parallelism for Distributed Training

ONE LOVE. ONE FUTURE.

Summary: operation parallelism

Data-parallel: ***???***

Model-parallel: ***???***

Summary: operation parallelism

Data-parallel: *one process applies all model on* ***partial data***

best for smaller model, more computations

Model-parallel: *one process applies* ***partial model*** *on all data*

best for larger model, fewer computations

Which one is better..

for word2vec?

In general?

Summary: operation parallelism

Data-parallel: *one process applies all model on **partial data***
best for smaller model, more computations

Model-parallel: *one process applies **partial model** on all data*
best for larger model, fewer computations

~~*Which one is better..*~~
~~*for word2vec?*~~
~~*In general?*~~

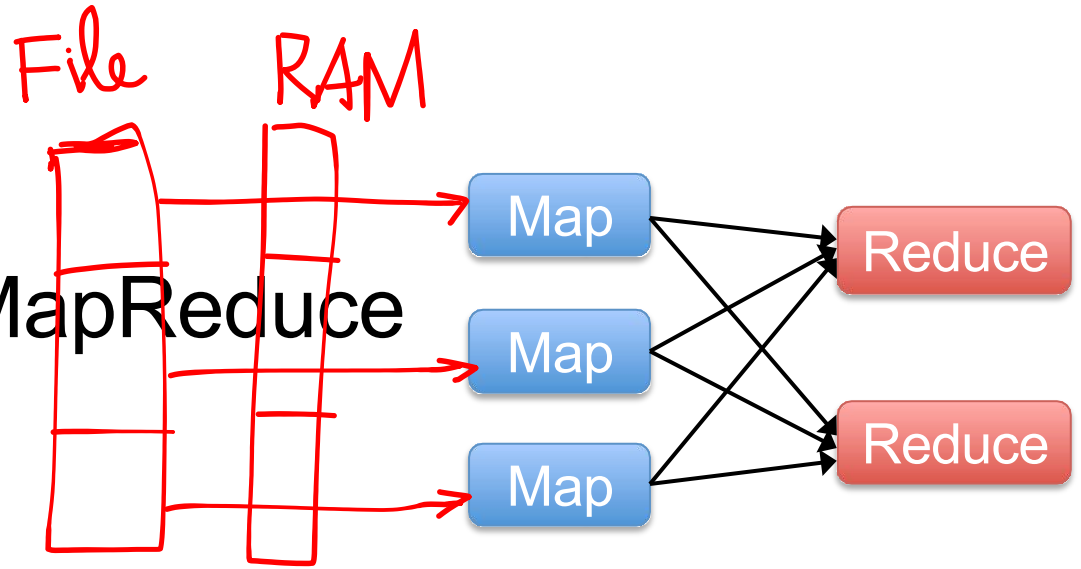
It depends…
- on model size
- on compute

Distributed Machine Learning on Spark

Data Flow Models

- Restrict the programming interface so that the system can do more automatically
- Express jobs as graphs of high-level operators
  - » System picks how to split each operator into tasks and where to run each task
- » Run parts twice fault recovery

Biggest example: MapReduce



Spark Computing Engine

- Extends a programming language with a distributed collection data-structure
- » “Resilient distributed datasets” (RDD)
- Open source at Apache
  - » Most active community in big data, with 50+ companies contributing
- Clean APIs in Java, Scala, Python Community: SparkR

Key Idea

- Resilient Distributed Datasets (RDDs)
  - » Collections of objects across a cluster with user controlled partitioning & storage (memory, disk, ...)
- » Built via parallel transformations (map, filter, …)
  - » The world only lets you make make RDDs such that they can be:

- Automatically rebuilt on failure

MLlib History

- MLlib is a Spark subproject providing machine learning primitives

- Initial contribution from AMPLab, UC Berkeley Shipped with Spark

since Sept 2013

MLlib: Available algorithms

- classification: logistic regression, linear SVM, naïve Bayes, least squares, classification tree
- regression: generalized linear models (GLMs), regression tree
- collaborative filtering: alternating least squares (ALS), non-negative matrix factorization (NMF)
- clustering: k-means
- decomposition: SVD, PCA
- optimization: stochastic gradient descent, L-BFGS

scikit-learn?

Algorithms:

- **classification:** SVM, nearest neighbors, random forest, …
- **regression:** support vector regression (SVR), ridge regression, Lasso, logistic regression, …
- **clustering:** k-means, spectral clustering, …
- **decomposition:** PCA, non-negative matrix factorization (NMF), independent component analysis (ICA), …

Mahout?

Algorithms:

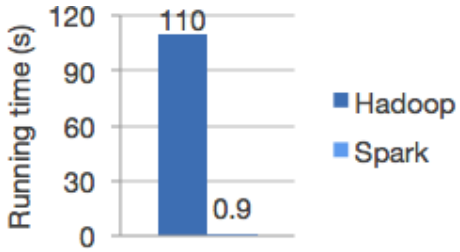
- **classification:** logistic regression, naive Bayes, random forest, …
- **collaborative filtering:** ALS, …
- **clustering:** k-means, fuzzy k-means, …
- **decomposition:** SVD, randomized SVD, …

Why MLlib?

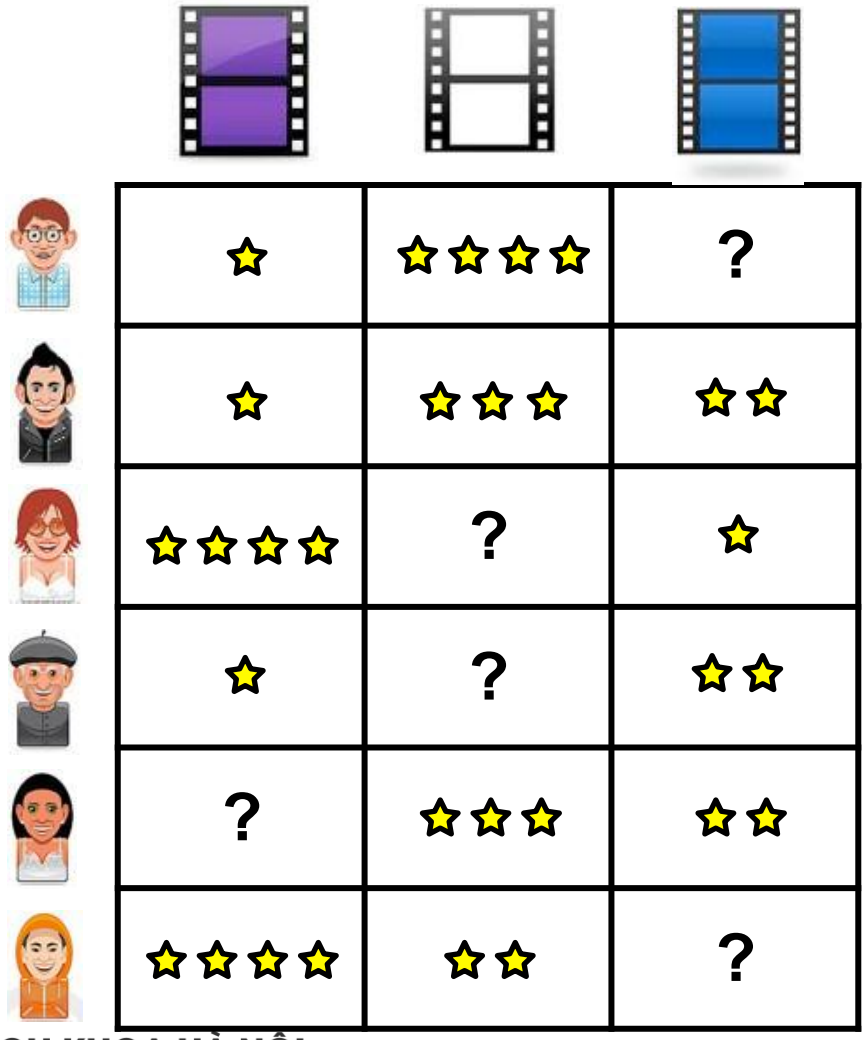
- It is built on Apache Spark, a fast and general engine for large-scale data processing.

- Run programs up to 100x faster than Hadoop MapReduce in memory, or 10x faster on disk.

- Write applications quickly in Java, Scala, or Python.



Collaborative filtering



- Recover a ra-ng matrix from a subset of its entries.

last·fm NETFLIX PANDORA

Linked in amazon.com

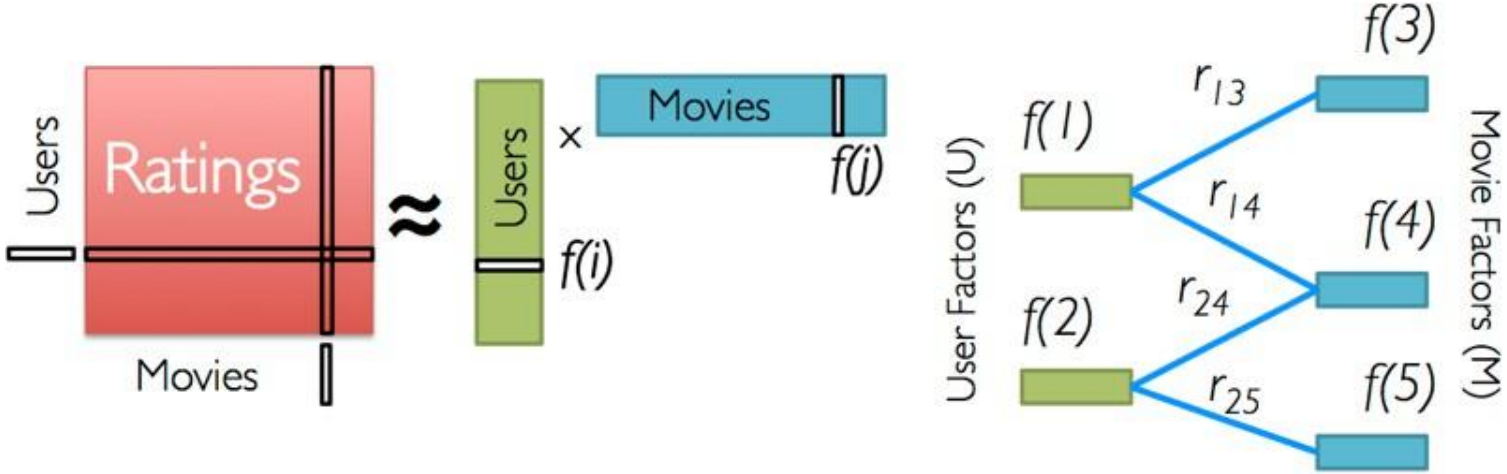
Collaborative filtering Example: ALS

```
// Load and parse the data
val data = sc.textFile("mllib/data/als/test.data")
val ratings = data.map(_.split(',') match {
    case Array(user, item, rate) =>
      Rating(user.toInt, item.toInt, rate.toDouble)
})

// Build the recommendation model using ALS
val model = ALS.train(ratings, 1, 20, 0.01)

// Evaluate the model on rating data
val usersProducts = ratings.map { case Rating(user, product, rate) =>
  (user, product)
}
val predictions = model.predict(usersProducts)
```

Implementation of ALS

- broadcast everything
- data parallel
- fully parallel

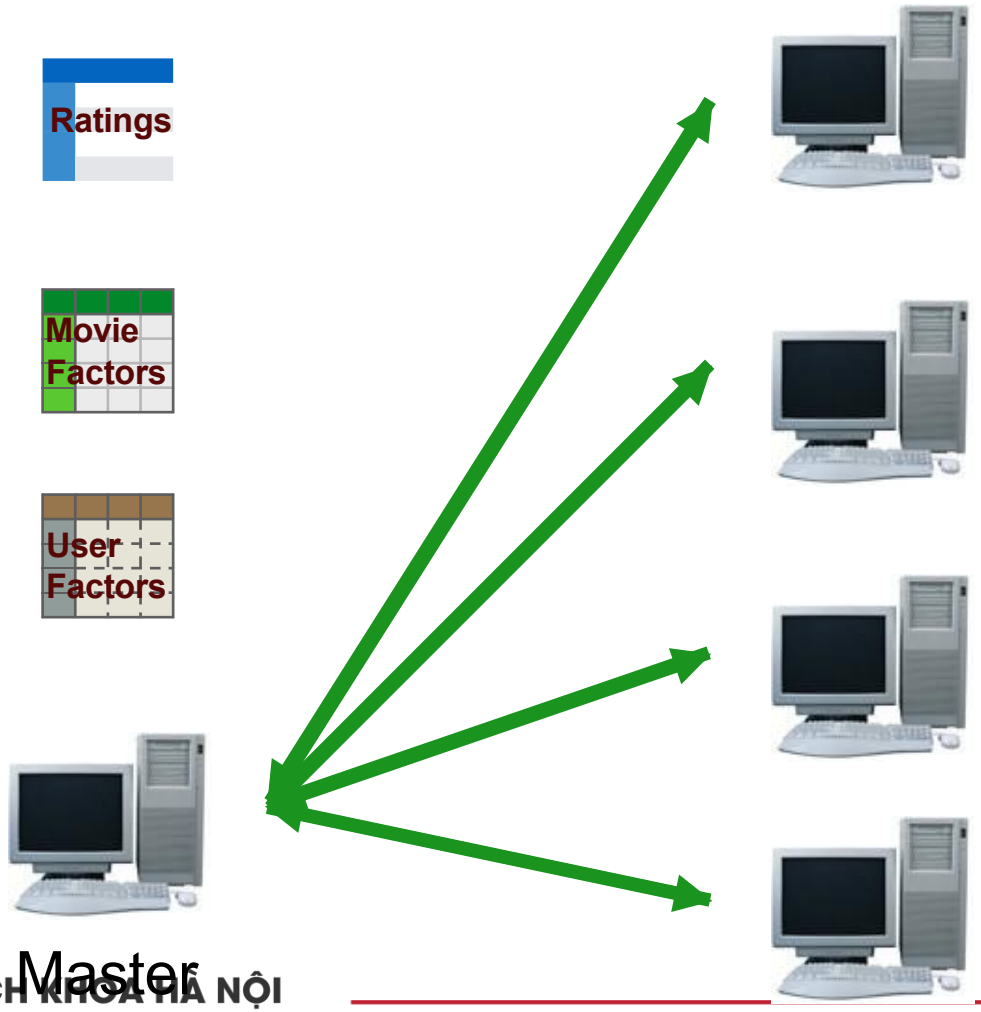
Alternating least squares (ALS)



Iterate:

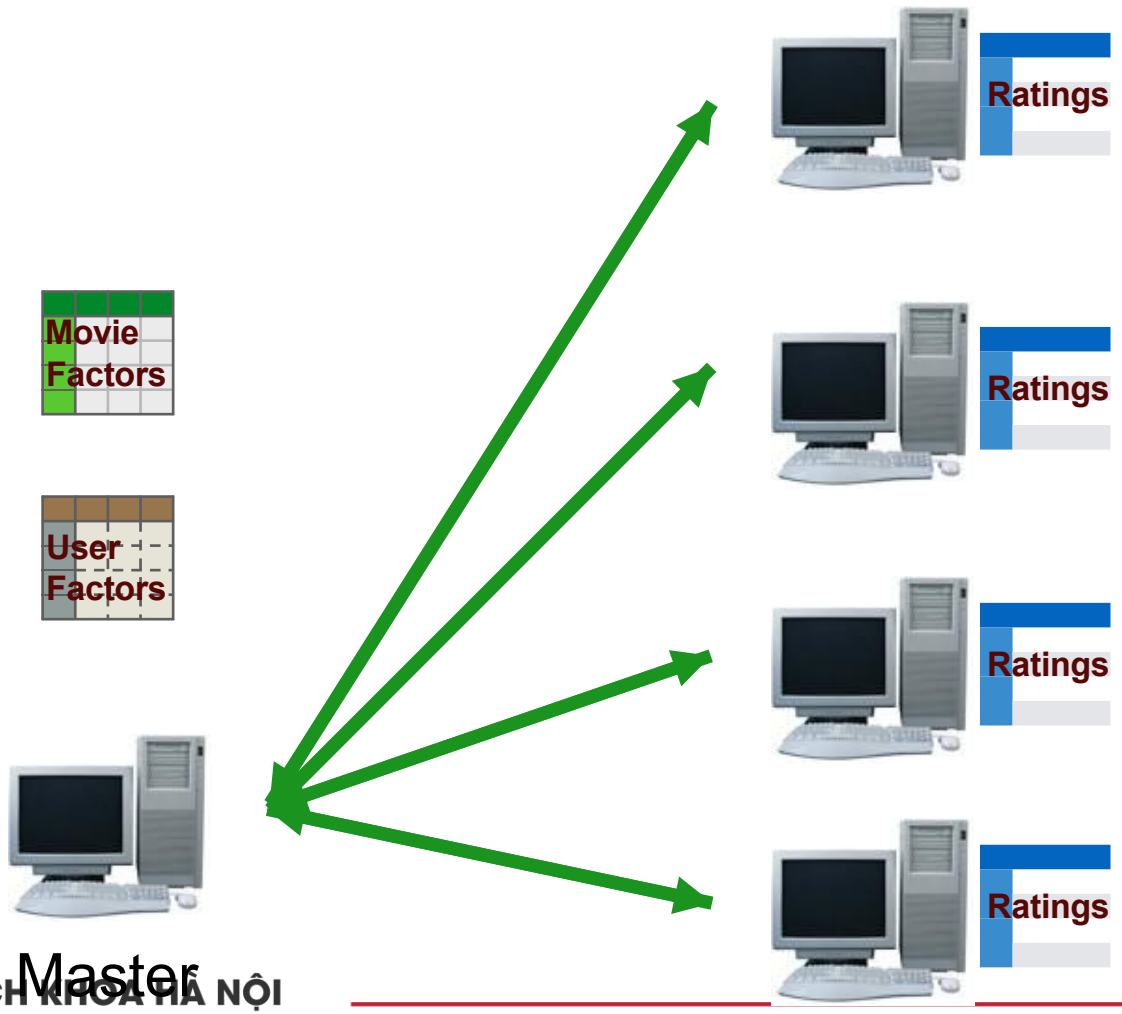
$$f[i] = \arg\min_{w \in \mathbb{R}^d} \sum_{j \in \text{Nbrs}(i)} \left(r_{ij} - w^T f[j]\right)^2 + \lambda ||w||_2^2$$

Broadcast everything



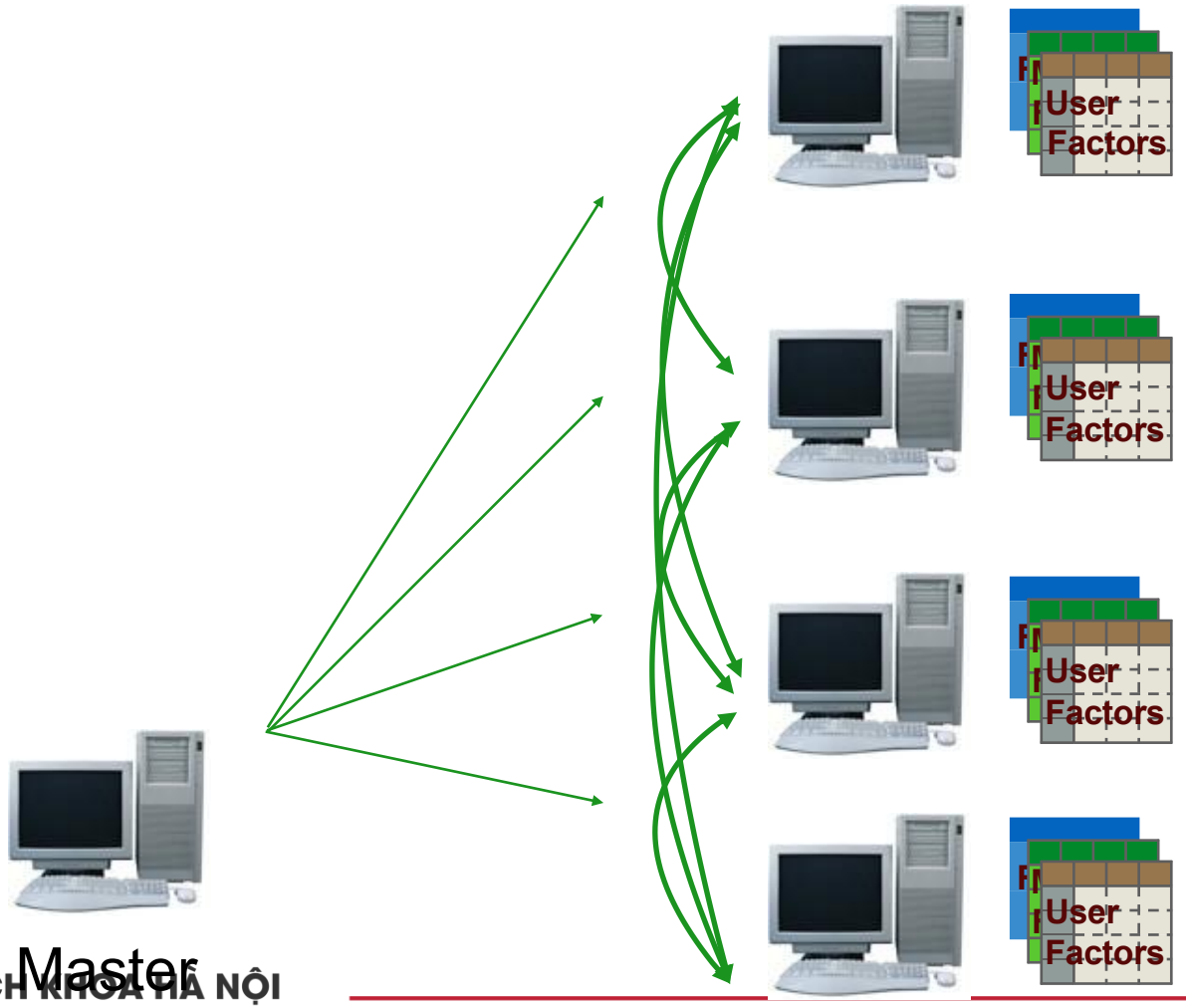
- Master loads (small) data file and initializes models.
- Master broadcasts data and initial models.
- At each iteration, updated models are broadcast again.
- Works OK for small data.
- Lots of communication overhead - doesn't scale well.

Data parallel



- Workers load data
- Master broadcasts initial models
- At each iteration, updated models are broadcast again
- Much better scaling
- Works on large datasets
- Works well for smaller models. (low K)

Fully parallel



- Workers load data
- Models are instantiated at workers.
- At each iteration, models are shared via join between workers.
- Much better scalability.
- Works on large datasets

ALS - wall-clock time

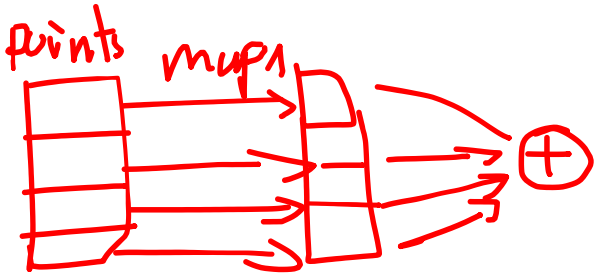
System	Wall–clock /me (seconds)
MATLAB	15443
Mahout	4206
GraphLab	291
MLlib	481

- Dataset: scaled version of Netflix data (9X in size).
- Cluster: 9 machines.
- MLlib is an order of magnitude faster than Mahout.
- MLlib is within factor of 2 of GraphLab.

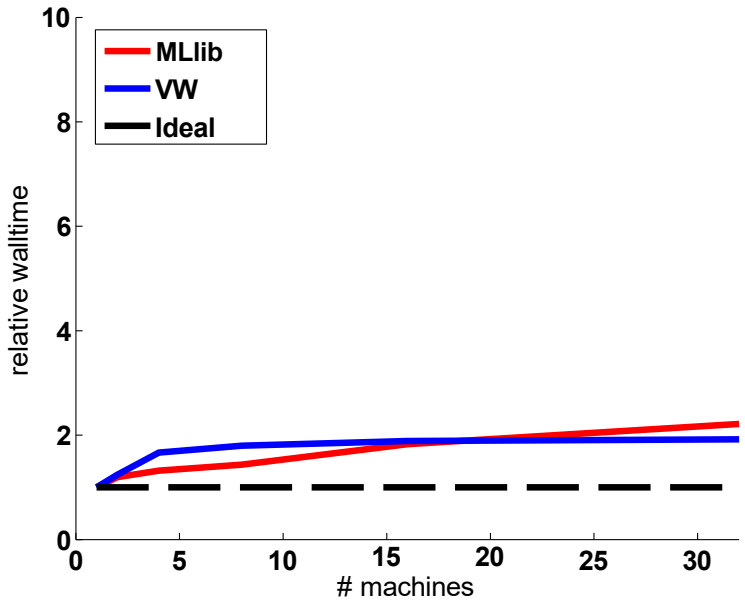
Gradient descent

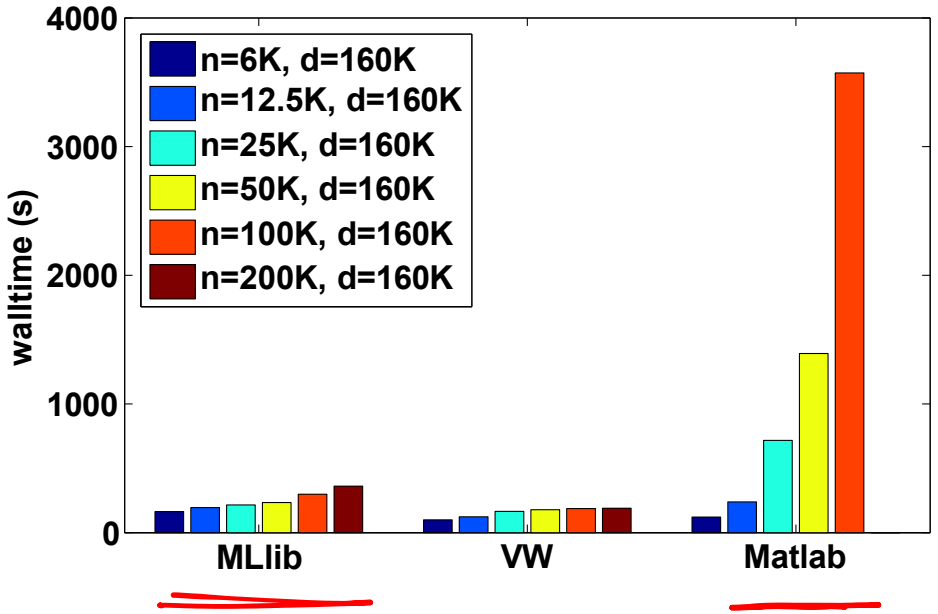
$$w \leftarrow w - \alpha \cdot \sum_{i=1}^{n} g(w; x_i, y_i)$$

```
val points = spark.textFile(...).map(parsePoint).cache()
var w = Vector.zeros(d)
for (i <- 1 to numIterations) {
  val gradient = points.map { p =>
    (1 / (1 + exp(-p.y * w.dot(p.x)) - 1) * p.y * p.x
  ).reduce(_ + _)
  w -= alpha * gradient
}
```



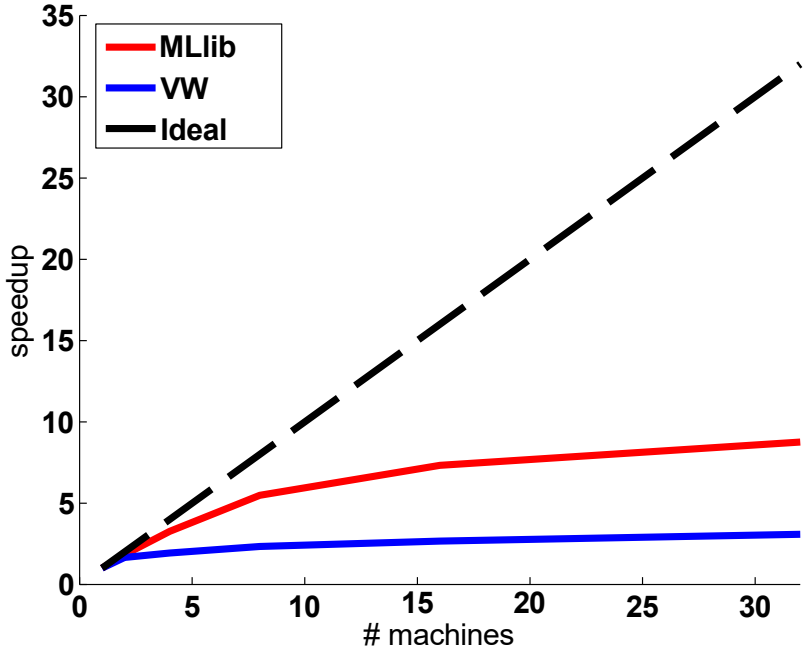
Logistic regression - weak scaling

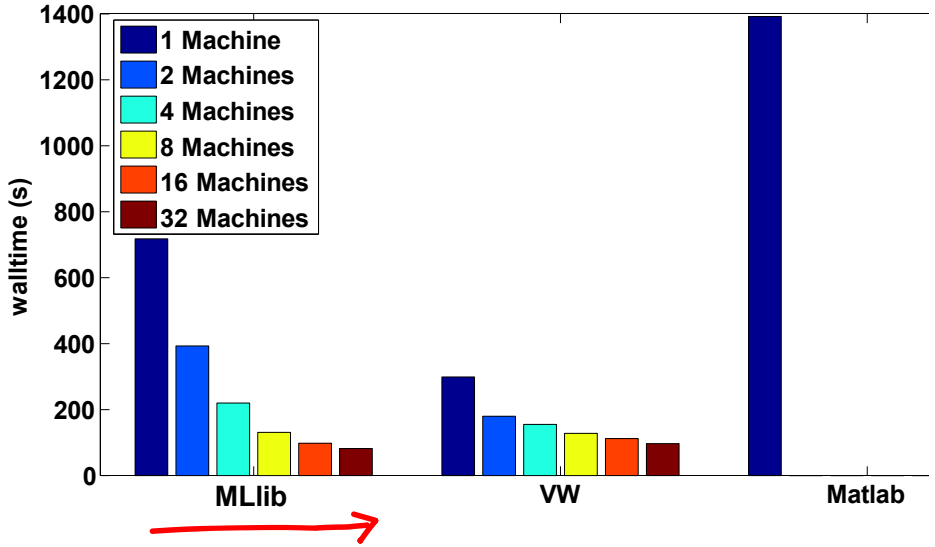




- Full dataset: 200K images, 160K dense features.
- Similar weak scaling.
- MLlib within a factor of 2 of VW's wall-clock time.

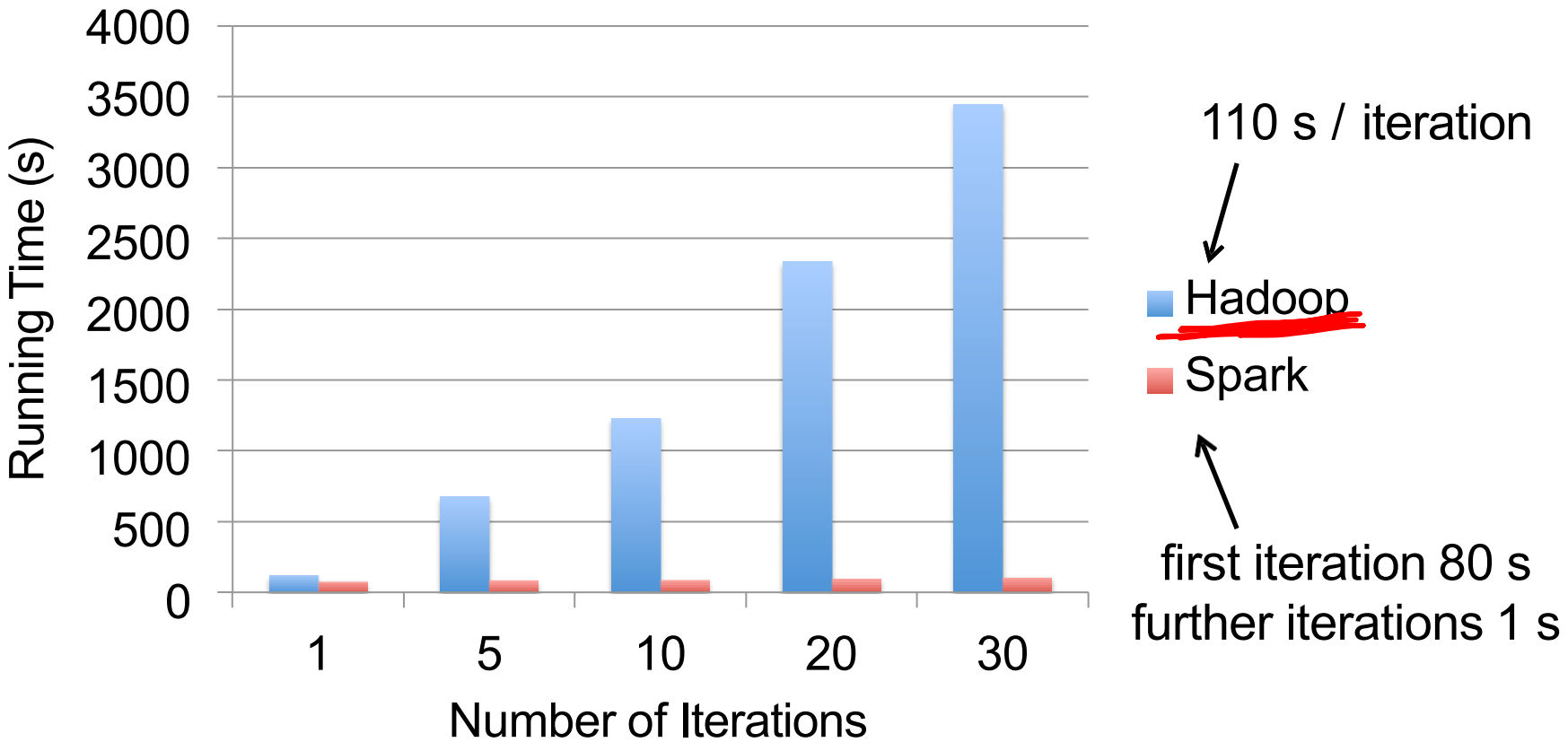
Logistic regression - strong scaling





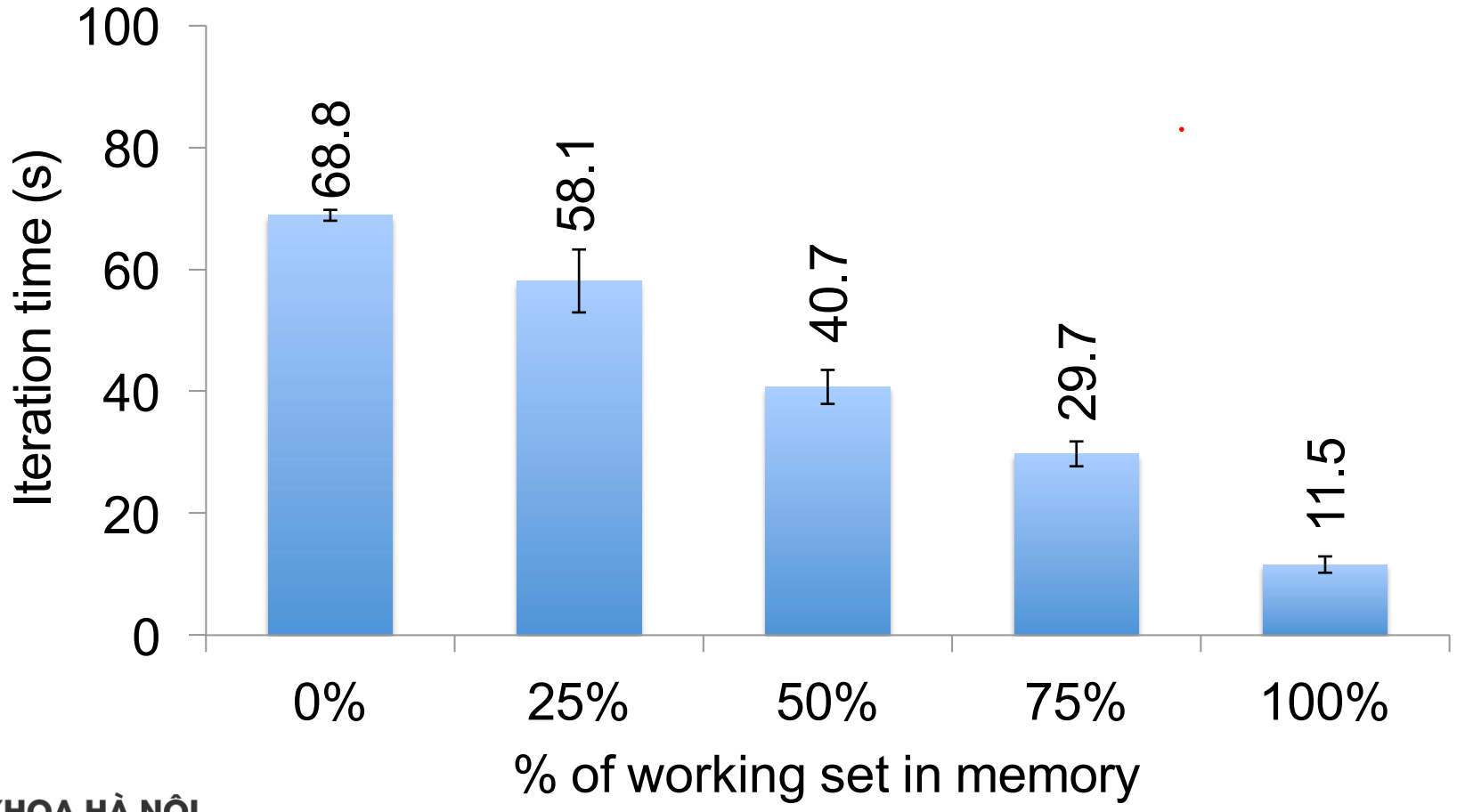
- Fixed Dataset: 50K images, 160K dense features.
- MLlib exhibits better scaling properties.
- MLlib is faster than VW with 16 and 32 machines.

Logistic Regression Results



100 GB of data on 50 m1.xlarge EC2 machines

Behavior with Less RAM

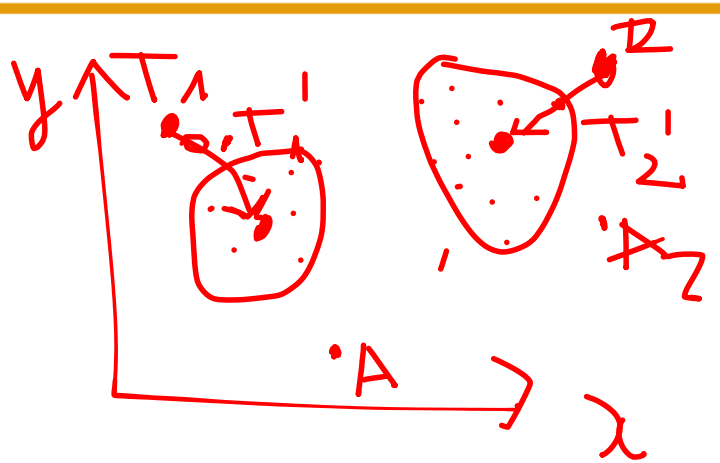


Implementation of k-means

Initialization:

- random
- k-means++
- k-means||

Implementation of k-means



Iterations:

- For each point, find its closest center.

$$l_i = \arg\min_j \|x_i - c_j\|_2^2$$

- Update cluster centers.

$$c_j = \frac{\sum_{i, l_i = j} x_j}{\sum_{i, l_i = j} 1}$$

Implementation of k-means

The points are usually sparse, but the centers are most likely to be dense. Computing the distance takes O(d) time. So the time complexity is O(n d k) per iteration. We don't take any advantage of sparsity on the running time. However, we have

$$\|x - c\|_2^2 = \|x\|_2^2 + \|c\|_2^2 - 2\langle x, c\rangle$$

Computing the inner product only needs non-zero elements. So we can cache the norms of the points and of the centers, and then only need the inner products to obtain the distances. This reduce the running time to O(nnz k + d k) per iteration.

However, is it accurate?

k-means (scala)

```scala
// Load and parse the data.
val data = sc.textFile("kmeans_data.txt")
val parsedData = data.map(_.split(' ').map(_.toDouble)).cache()

// Cluster the data into two classes using KMeans.
val clusters = KMeans.train(parsedData, 2, numIterations = 20)

// Compute the sum of squared errors.
val cost = clusters.computeCost(parsedData)
println("Sum of squared errors = " + cost)
```

k-means (python)

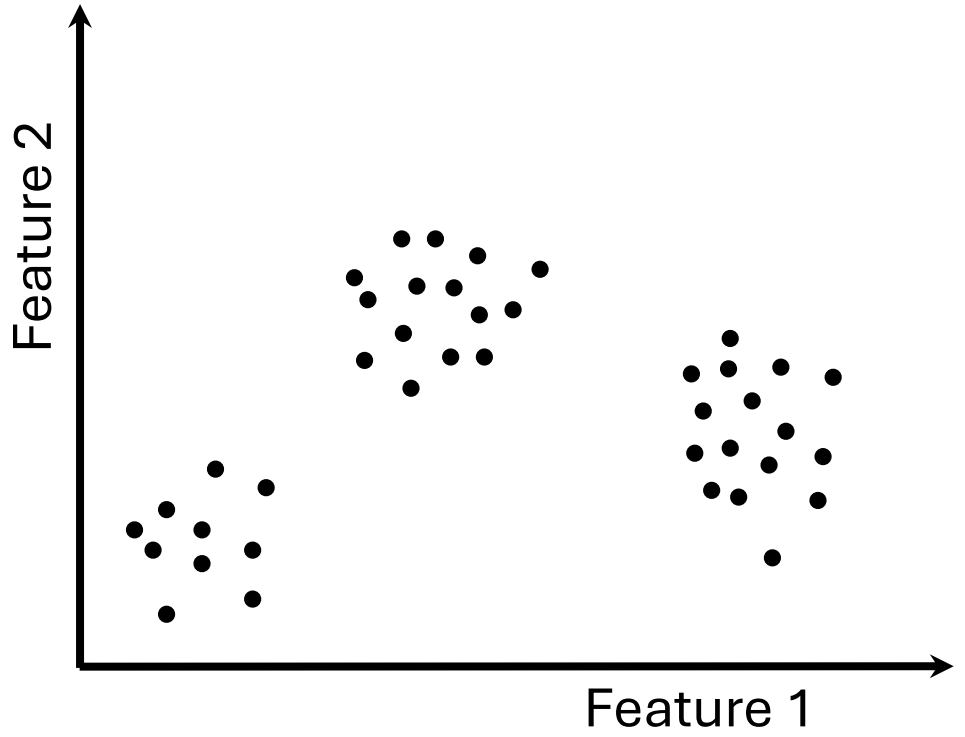
```
# Load and parse the data
data = sc.textFile("kmeans_data.txt")
parsedData = data.map(lambda line:
                array([float(x) for x in line.split('   ')])).cache()

# Build the model (cluster the data)
clusters = KMeans.train(parsedData, 2, maxIterations = 10,
              runs = 1, initialization_mode = "kmeans||")

# Evaluate clustering by computing the sum of squared errors
def error(point):
    center = clusters.centers[clusters.predict(point)]
    return sqrt(sum([x**2 for x in (point - center)]))

cost = parsedData.map(lambda point: error(point))
         .reduce(lambda x, y: x + y)
print("Sum of squared error = " + str(cost))
```

Dimension reduction (PCA) + k-means

```
// compute principal components
val points: RDD[Vector] = ...
val mat = RowRDDMatrix(points)
val pc = mat.computePrincipalComponents(20)

// project points to a low-dimensional space
val projected = mat.multiply(pc).rows

// train a k-means model on the projected data
val model = KMeans.train(projected, 10)
```

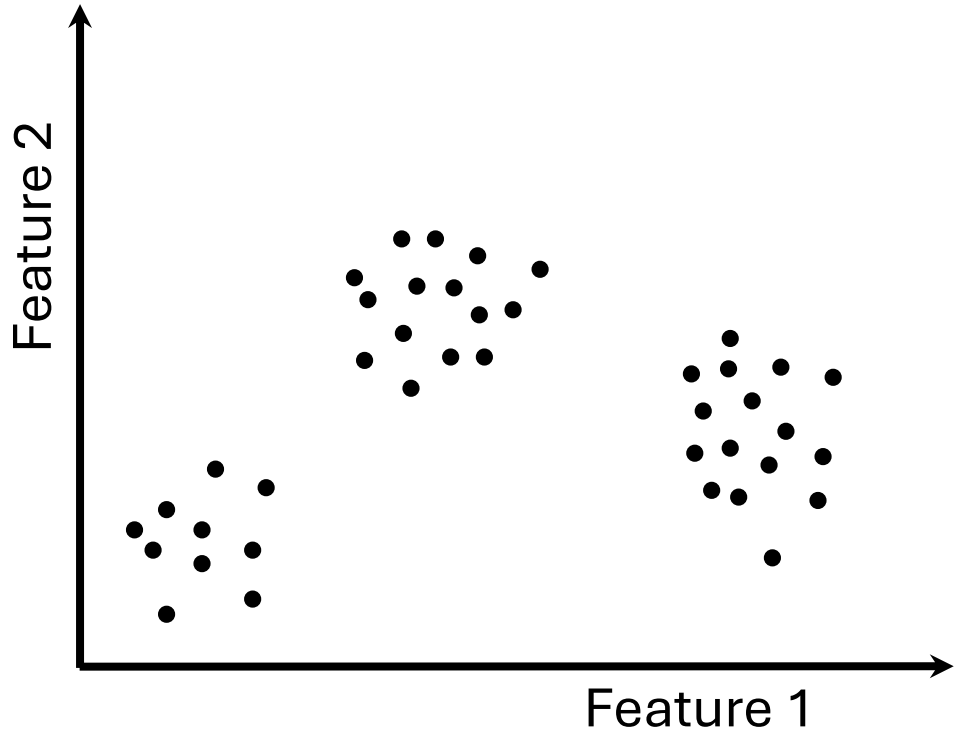
K-MEANS ALGORITHM

- Initialize K cluster centers
- Repeat until convergence:
  - Assign each data point to the cluster with the closest center.
  - Assign each cluster center to be the mean of its cluster’s data points.



K-MEANS ALGORITHM

- Initialize K cluster centers
- Repeat until convergence:
  - Assign each data point to the cluster with the closest center.
  - Assign each cluster center to be the mean of its cluster's data points.

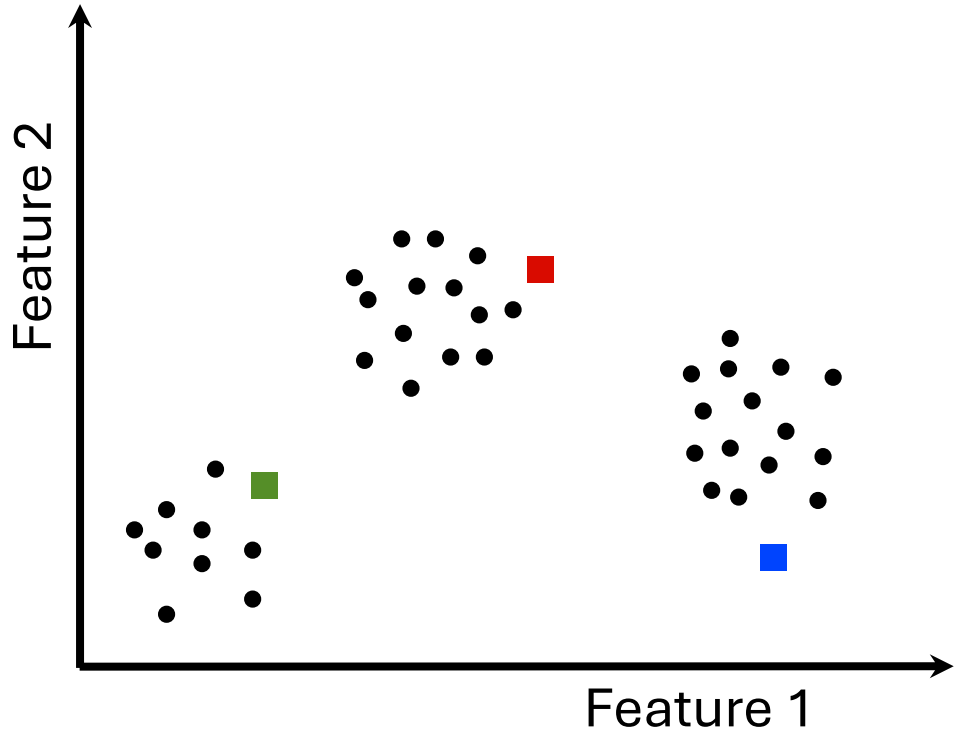


K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:
  - Assign each data point to the cluster with the closest center.
  - Assign each cluster center to be the mean of its cluster's data points.

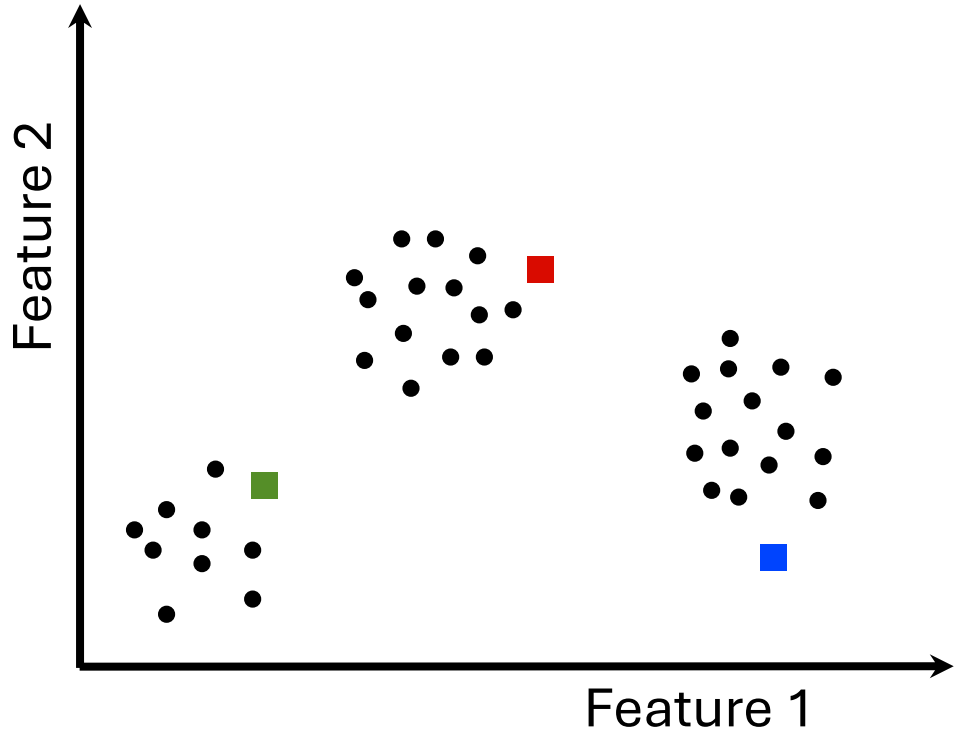


K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:
  - Assign each data point to the cluster with the closest center.
  - Assign each cluster center to be the mean of its cluster’s data points.

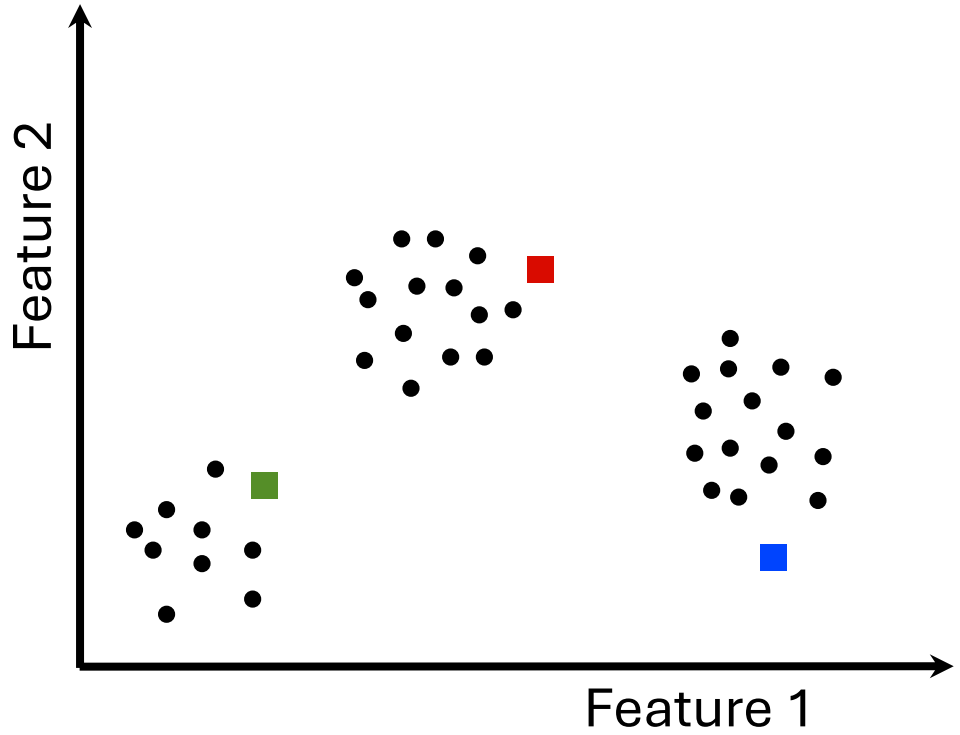


K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:
  - Assign each data point to the cluster with the closest center.
  - Assign each cluster center to be the mean of its cluster’s data points.



K-MEANS ALGORITHM

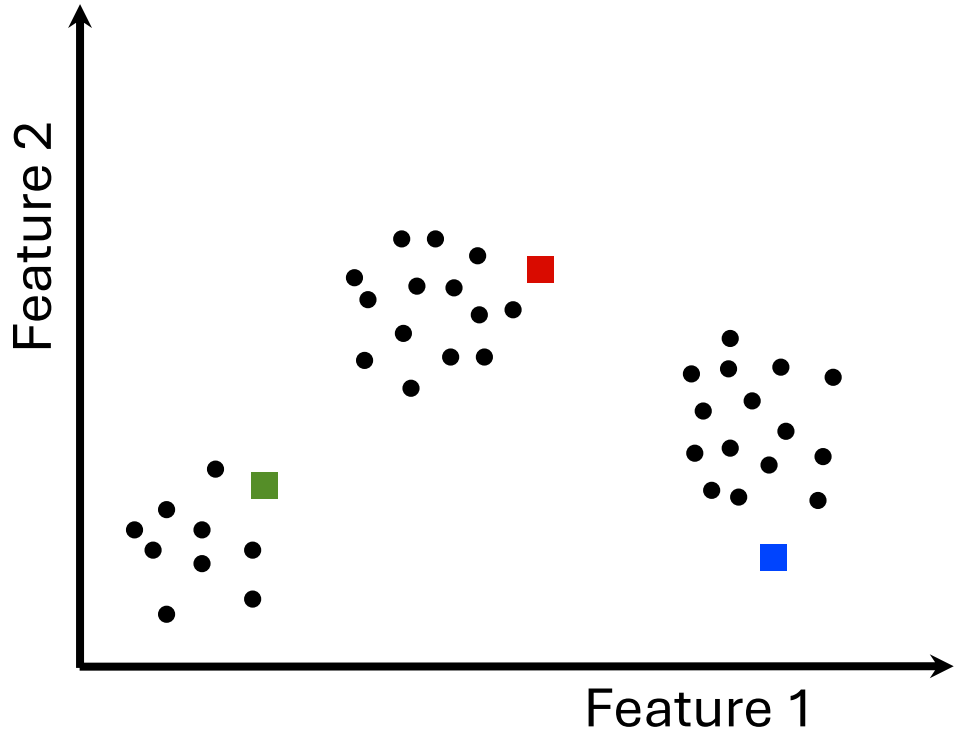
- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:

```
closest = data.map(p =>

(closestPoint(p,centers),p))
```

Assign each cluster center to be the mean of its cluster's data points.



K-MEANS ALGORITHM

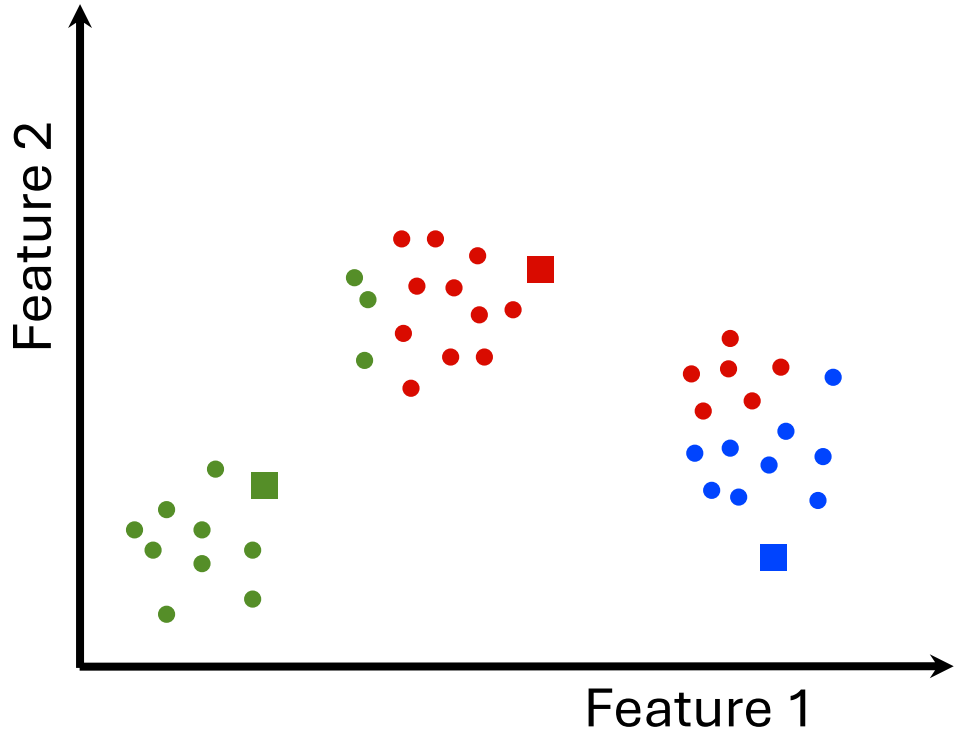
- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:

```
closest = data.map(p =>

(closestPoint(p,centers),p))
```

Assign each cluster center to be the mean of its cluster's data points.



K-MEANS ALGORITHM

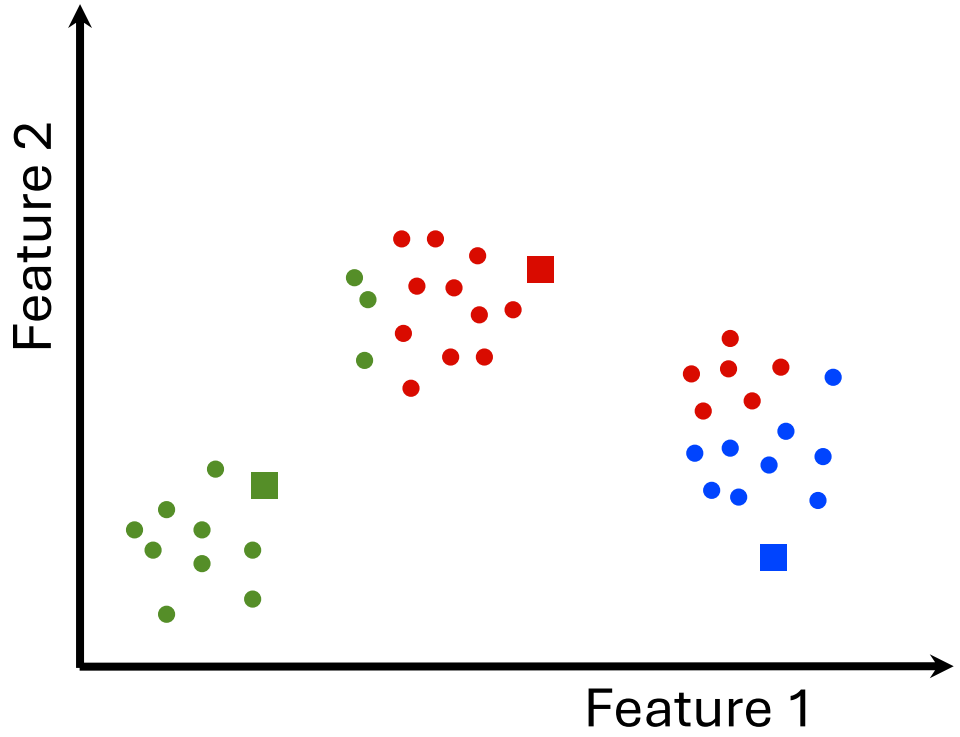
- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:

```
closest = data.map(p =>

(closestPoint(p,centers),p))
```

Assign each cluster center to be the mean of its cluster's data points.



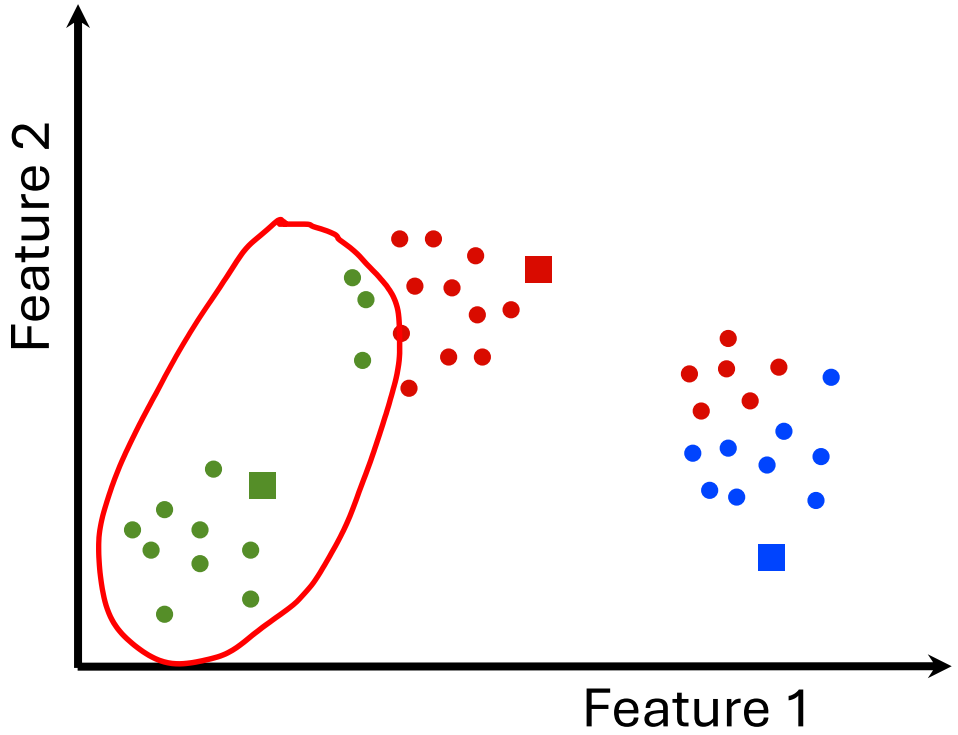
K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:

```
closest = data.map(p =>

(closestPoint(p,centers),p))

pointsGroup =
    closest.groupByKey()
```



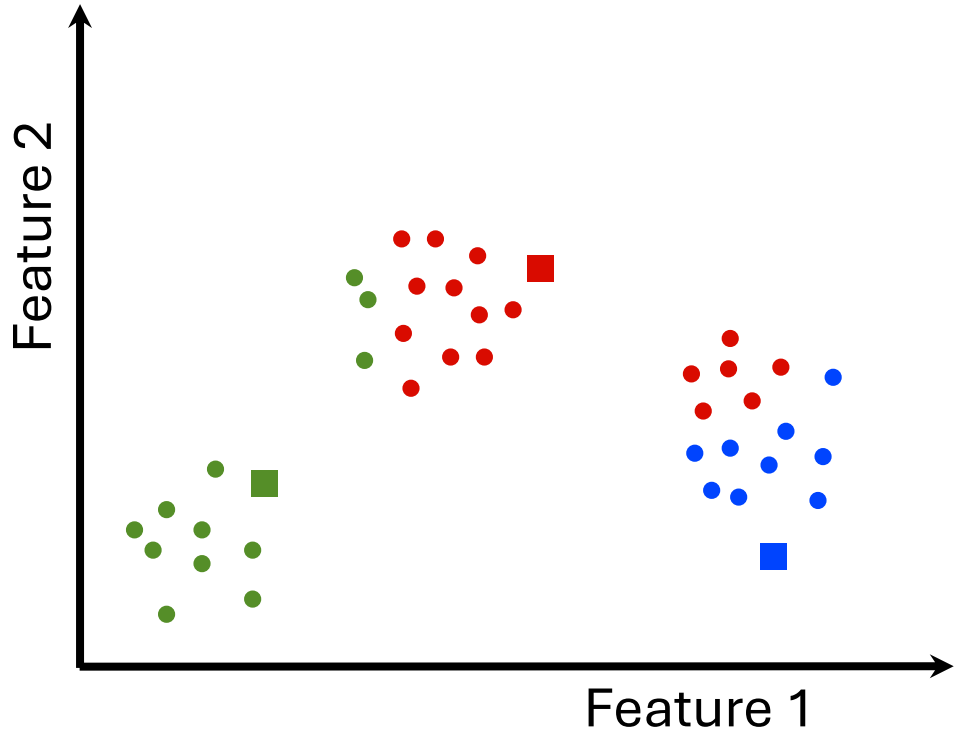
K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:

```
closest = data.map(p =>

(closestPoint(p,centers),p))

pointsGroup =
    closest.groupByKey()

newCenters = pointsGroup.mapValues(
    ps => average(ps))
```



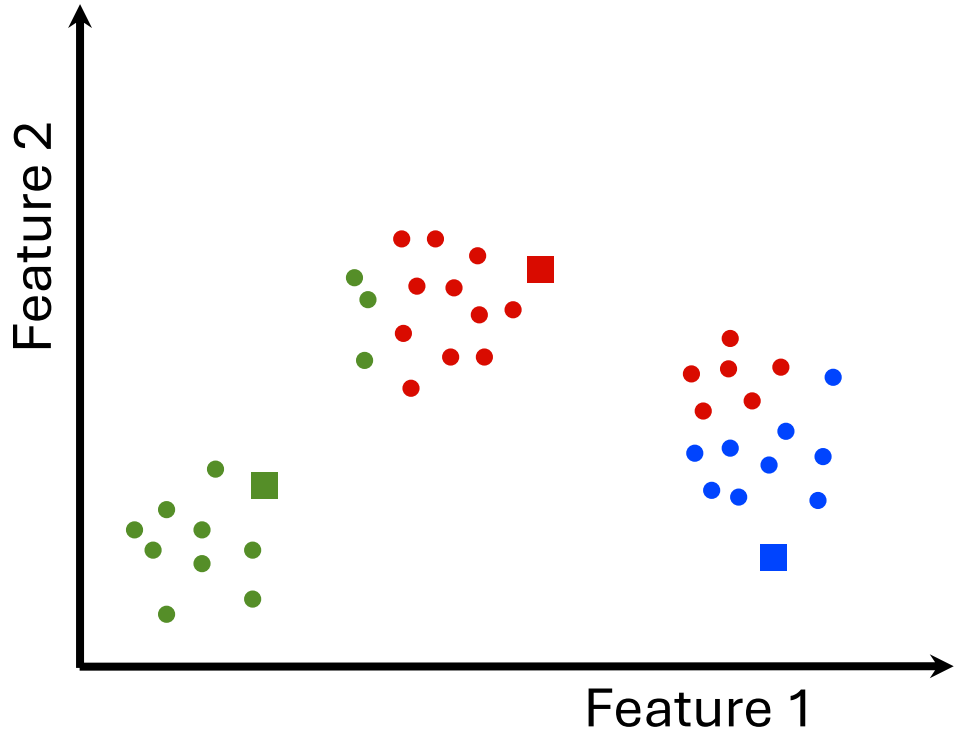
K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:

```
closest = data.map(p =>

(closestPoint(p,centers),p))

pointsGroup =
    closest.groupByKey()

newCenters = pointsGroup.mapValues(
    ps => average(ps))
```



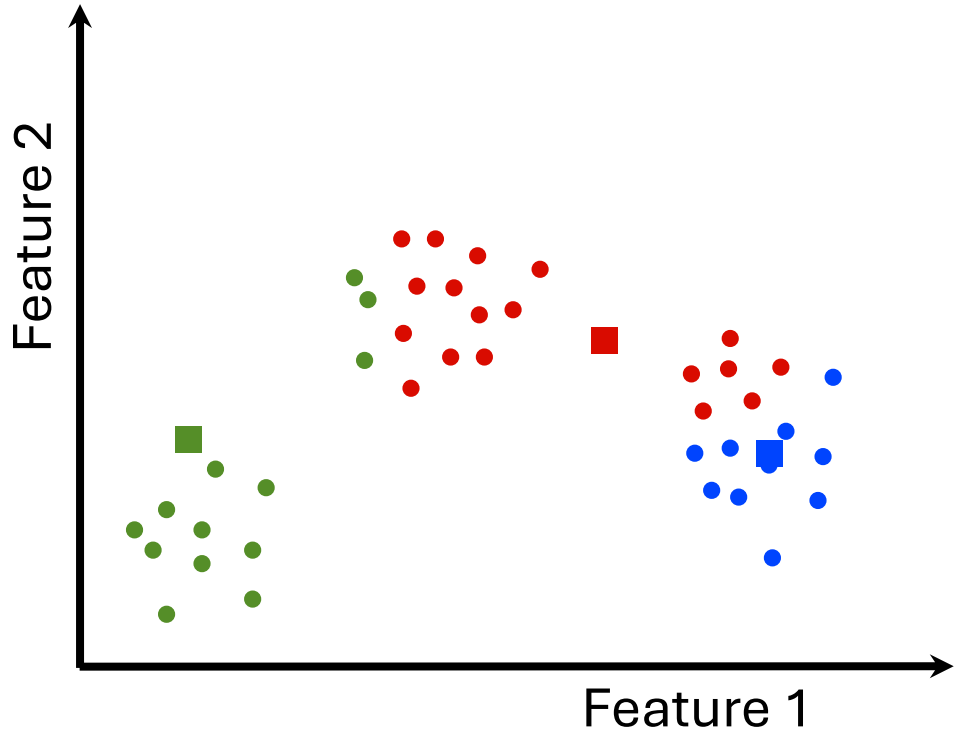
K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:

```
closest = data.map(p =>
    (closestPoint(p,centers),p))
pointsGroup =
    closest.groupByKey()
newCenters = pointsGroup.mapValues(
    ps => average(ps))
```



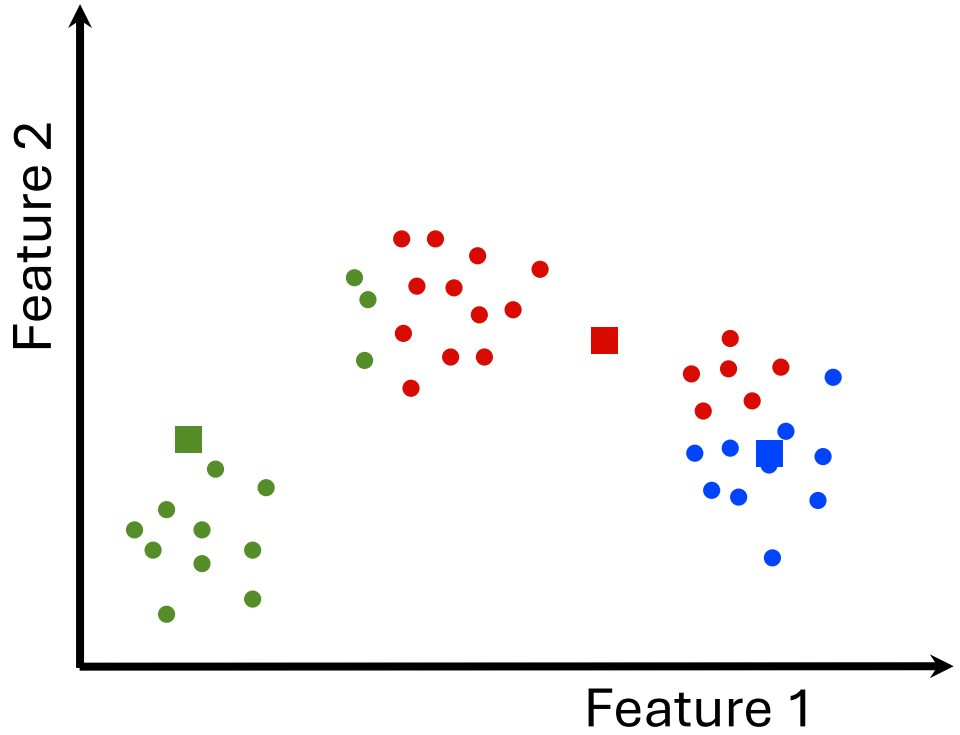
K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

- Repeat until convergence:

```
while (dist(centers, newCenters) > ε)
closest = data.map(p =>
    (closestPoint(p,centers),p))
pointsGroup =
    closest.groupByKey()
newCenters =pointsGroup.mapValues(
    ps => average(ps))
```



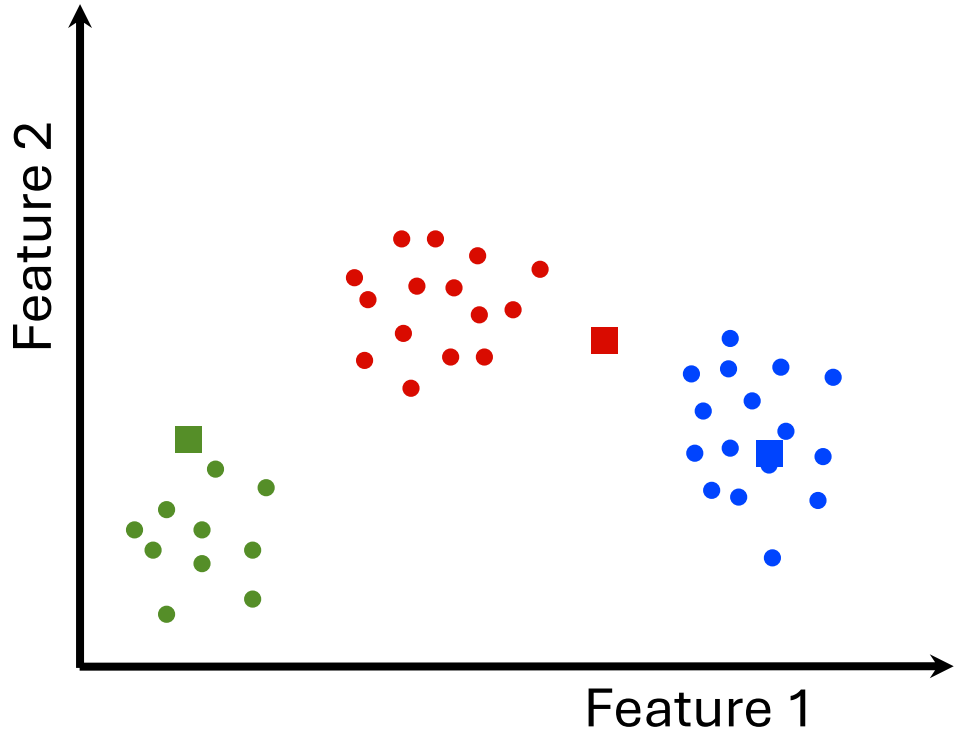
K-MEANS ALGORITHM

- Initialize K cluster centers

```
centers = data.takeSample(
    false, K, seed)
```

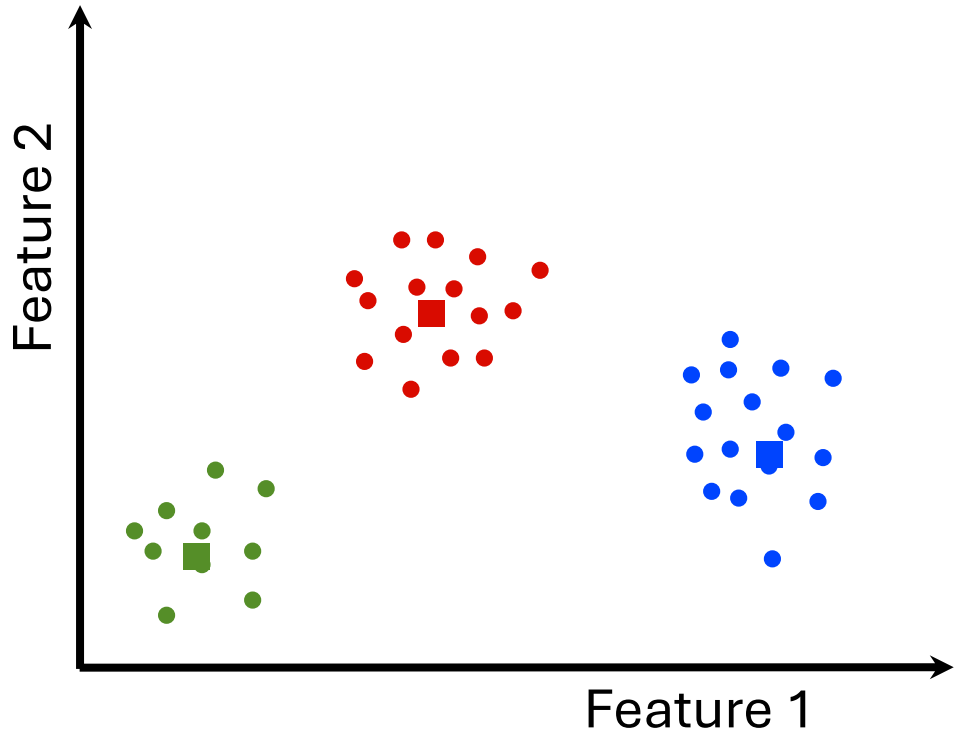
- Repeat until convergence:

```
while (dist(centers, newCenters) > ε)
  closest = data.map(p =>
      (closestPoint(p,centers),p))
  pointsGroup =
      closest.groupByKey()
  newCenters =pointsGroup.mapValues(
      ps => average(ps))
```



K-MEANS ALGORITHM

```
centers = data.takeSample(
    false, K, seed)
while (d > ε)
{
 closest = data.map(p =>
     (closestPoint(p,centers),p))
 pointsGroup =
     closest.groupByKey()
 newCenters =pointsGroup.mapValues(
     ps => average(ps))
 d = distance(centers, newCenters)
 centers = newCenters.map(_)
}
```



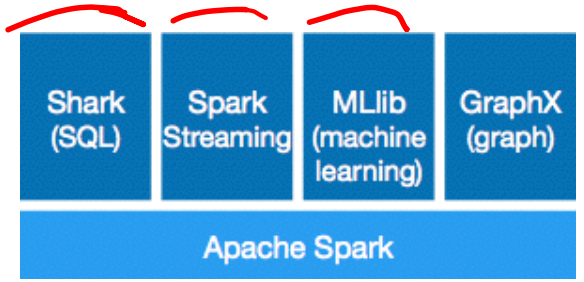
Why MLlib?

- Spark is a general-purpose big data platform.
- Runs in standalone mode, on YARN, EC2, and Mesos, also on Hadoop v1 with SIMR.
- Reads from HDFS, S3, HBase, and any Hadoop data source.
- MLlib is a standard component of Spark providing machine learning primitives on top of Spark.

- MLlib is also comparable to or even better than other libraries specialized in large-scale machine learning.

Why MLlib?

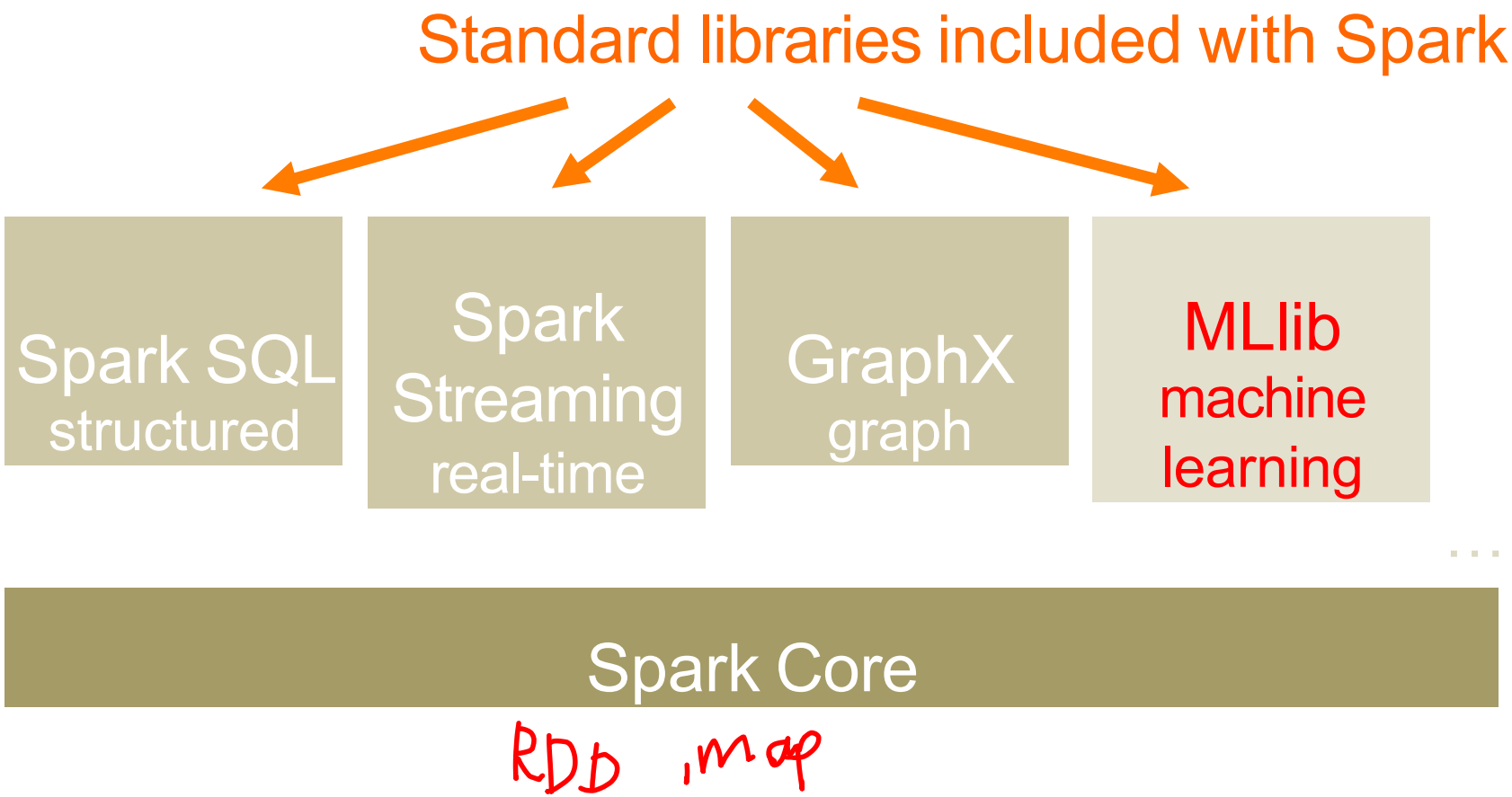
- It ships with Spark as a standard component.



Why MLlib?

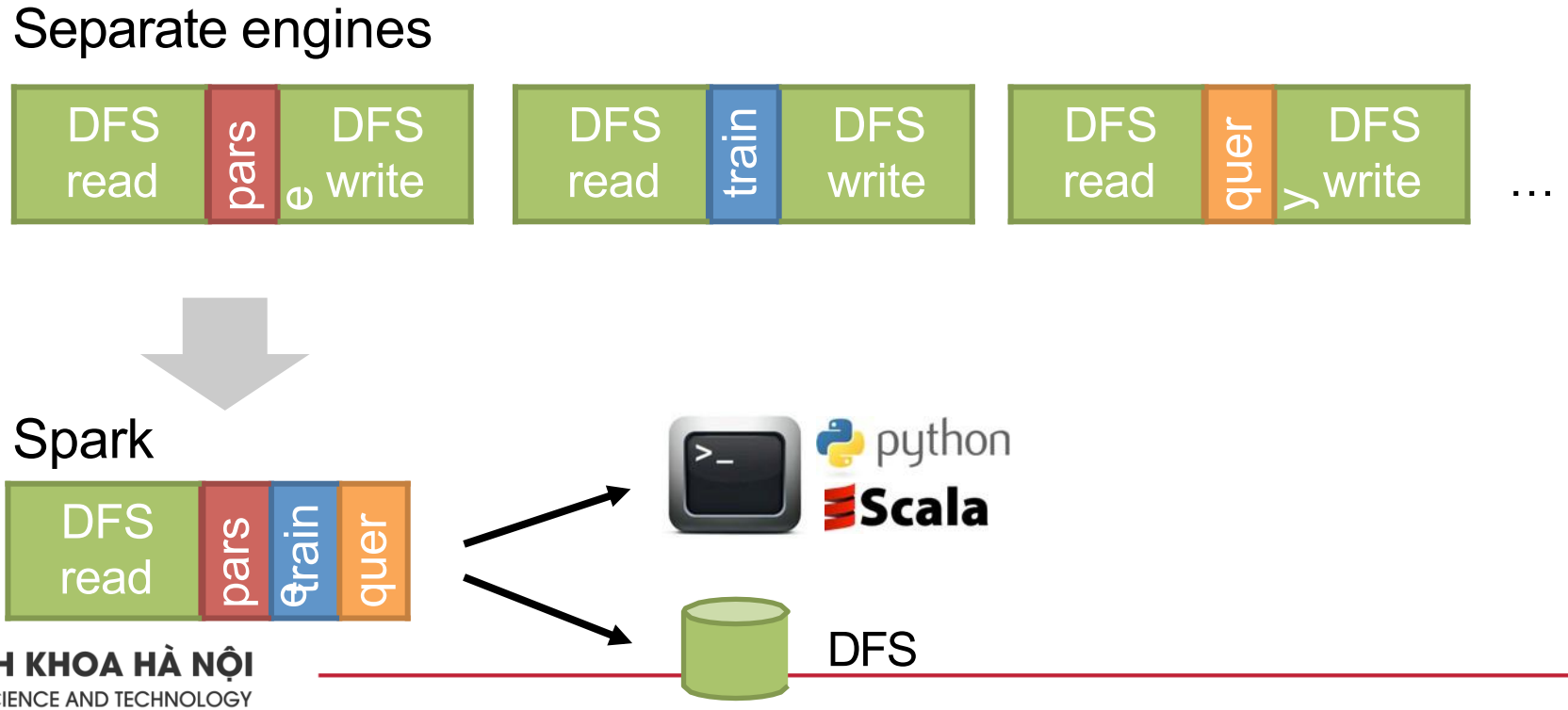
- Scalability
- Performance
- User-friendly APIs
- Integration with Spark and its other components

A General Platform



Benefit for Users

Same engine performs data extraction, model training and interactive queries



Spark SQL + MLlib

```
// Data can easily be extracted from existing sources,
// such as Apache Hive.
val trainingTable = sql("""
  SELECT e.action,
         u.age,
         u.latitude,
         u.longitude
  FROM Users u
  JOIN Events e
  ON u.userId = e.userId""")

// Since `sql` returns an RDD, the results of the
above
// query can be easily used in MLlib.
val training = trainingTable.map { row =>
  val features = Vectors.dense(row(1), row(2),
  row(3)) LabeledPoint(row(0), features)
}

val model = SVMWithSGD.train(training)
```

Streaming + MLlib

```
// collect tweets using streaming

// train a k-means model
val model: KMmeansModel = ...

// apply model to filter tweets
val tweets = TwitterUtils.createStream(ssc, Some(authorizations(0)))
val statuses = tweets.map(_.getText)
val filteredTweets =
  statuses.filter(t => model.predict(featurize(t)) == clusterNumber)

// print tweets within this particular cluster
filteredTweets.print()
```

GraphX + MLlib

```
// assemble link graph
val graph = Graph(pages, links)
val pageRank: RDD[(Long, Double)] = graph.staticPageRank(10).vertices

// load page labels (spam or not) and content features
val labelAndFeatures: RDD[(Long, (Double, Seq((Int, Double))))] = ...
val training: RDD[LabeledPoint] =
  labelAndFeatures.join(pageRank).map {
    case (id, ((label, features), pageRank)) =>
      LabeledPoint(label, Vectors.sparse(features ++ (1000, pageRank))
}

// train a spam detector using logistic regression
val model = LogisticRegressionWithSGD.train(training)
```